

Travaux Dirigés de Compilation n°10

Licence d'informatique

Traduction des tableaux en assembleur

L'objectif de ce TD est d'appliquer la traduction des tableaux en assembleur.

► Exercice 1. *Tableau de caractères à une dimension*

Implémentez en `nasm` l'équivalent d'un tableau de 64 caractères indicés de 0 à 63 et codés sur 1 octet chacun. Réservez la mémoire pour ce tableau dans une section `.data` ou `.bss`. Implémentez l'affichage de chaque case du tableau en la copiant dans un registre et en appelant `show_registers`. Pour l'accès mémoire dans le tableau, utilisez la syntaxe `[registre1 + registre2 * largeur]`, où :

- `registre1` est l'adresse de base du tableau,
- `registre2` est l'indice d'une case du tableau,
- `largeur` est la taille en octets d'une case du tableau (donc ici 1).

► Exercice 2. *Tableau d'entiers indicés de 32 à 212*

Mêmes questions et mêmes consignes, mais pour un tableau d'entiers indicés de 32 à 212 et codés sur 4 octets chacun. (Si vous initialisez la case d'indice f avec $(f - 32) * 5/9$, votre tableau traduit la température des degrés Fahrenheit en degrés Celsius.)

► Exercice 3. *Tableau à deux dimensions*

Mêmes questions et mêmes consignes, mais pour un tableau de pointeurs codés sur 8 octets chacun, avec deux dimensions, comme dans :

```
void *my_array[6][9];
```

Les indices de la première dimension iront de 0 à 5, et ceux de la deuxième dimensions de 0 à 8, comme en C. Pour l'accès mémoire dans le tableau, utilisez la syntaxe `[registre1 + registre2 * largeur]`, où `registre2` est l'indice d'une case du tableau unidimensionnel équivalent. Pour calculer `registre2`, vous pouvez utiliser la pile (8 octets pour l'indice de la première dimension, 8 octets pour celui de la deuxième dimension) ou deux registres supplémentaires.